

ENTRADA Y SALIDA ESTÁNDAR

PROFESOR: Federico Melo Barrero

CÓMO FUNCIONAN LA ENTRADA Y LA SALIDA ESTÁNDAR

Todo programa que escriba para este curso debe leer desde la entrada estándar (*standard input*) y escribir en la salida estándar (*standard output*). No debe recibir argumentos por línea de comandos ni consultar fuentes externas.

FORMATO DE LOS ARCHIVOS DE ENTRADA Y SALIDA

Por convención, las entradas se escriben en un archivo de texto plano con extensión `.in` y las salidas en uno con extensión `.out`.

La estructura general del archivo de entrada es uniforme: la primera línea contiene un entero T , la cantidad de casos de prueba. A continuación siguen T bloques, uno por caso de prueba, cuyo formato específico está definido en cada problema. El archivo de salida debe tener una respuesta para cada uno de los T casos, en el orden en el que están en el archivo de entrada, siguiendo el formato especificado por el problema.

Los calificadores de las soluciones son automáticos; en caso de que se transgreda el formato de entrada y salida, se otorgará a la implementación un puntaje de 0.

REDIRECCIÓN Y EJECUCIÓN

Para probar su programa en la consola, `<` redirige el contenido de un archivo hacia la entrada estándar y `>` redirige la salida estándar hacia un archivo.

Supóngase una implementación en un archivo llamado `solution`, con los casos de prueba en `input.in` y la salida esperada en `output.out`:

Python

```
1 python solution.py < input.in > output.out
```

Java

Tras compilar el código (e.g. `javac solution.java`):

```
1 java solution < input.in > output.out
```

C++

Tras compilar el código (e.g. `g++ solution.cpp -o solution`):

```
1 ./solution < input.in > output.out
```

JavaScript

```
1 node solution.js < input.in > output.out
```

Redirigir un archivo de entrada no es estrictamente necesario; también se pueden digitar los casos de prueba a mano en la consola (el programa se queda esperando la entrada estándar, tal como cuando se ejecuta sin <). Sin embargo, para probar varios casos o casos grandes, lo más cómodo es usar redirección.

EJEMPLO: PROBLEMA P0

El Problema P0 es un problema de juguete que ilustra, con un enunciado sencillo, cómo se espera que se lean los datos de entrada y se escriban las respuestas.

ENUNCIADO

Entrada: La primera línea contiene la cantidad de casos de prueba T . Cada una de las siguientes T líneas contiene una lista de n números enteros ($0 < n \leq 1000$), separados por espacio.

Salida: Por cada caso de prueba se espera una línea de la forma $cp \ sp$, donde cp es la cantidad de enteros pares leídos y sp es la suma de esos enteros pares.

CASO DE EJEMPLO

P0.in (entrada):

```
1 6
2 1 2 3 4 5 6 7 8 9 10
3 1 -2 3 -4 5 -6 7 -8 9 -10
4 2 4 6 8
5 1 3 5 7
6 1 1 1 1 1
7 -2 2 -2 2 -2 2
```

P0.out (salida esperada):

```
1 5 30
2 5 -30
3 4 20
4 0 0
5 0 0
6 6 0
```

EJECUCIÓN

Con los casos de prueba anteriores guardados en P0.in:

Python

```
1 python ProblemaP0.py < P0.in > P0.out
```

Java

```
1 java ProblemaP0 < P0.in > P0.out
```

C++

```
1 ./ProblemaP0 < P0.in > P0.out
```

JavaScript

```
1 node ProblemaP0.js < P0.in > P0.out
```